Predicting a Winning Seasons

Vanhi Chava

2026-02-16

```r
# libraries
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
library(tidyverse)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3

## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v forcats   1.0.0     v readr     2.1.5
## v ggplot2   4.0.2     v stringr   1.5.1
## v lubridate 1.9.4     v tibble    3.2.1
## v purrr     1.0.2     v tidyr     1.3.1

## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(glmnet)
```

```
## Loading required package: Matrix
##
## Attaching package: 'Matrix'
##
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
##
## Loaded glmnet 4.1-10
```

```r
library(rpart)
library(rpart.plot)
```

```
## Warning: package 'rpart.plot' was built under R version 4.4.3
```

```r
library(ggplot2)
```

Intrinsic Interpretability Portfolio

Background

Apply two intrinsic interpretability methods to a problem of your choice. The essential task: develop critical judgment about what works through direct experience.

a) Problem Formulation

Background

Specify:

- The problem context and dataset
- Why interpretability matters here (not just: "it would be nice' ')
- Who needs the interpretation and what decisions it enables
- Existing interpretable ML approaches for this data type (brief overview)

Solution

For this assignment, I will be working with advanced team statistics of NBA data (2025-2026 season) which was obtained from Basketball Reference. Since there are 30 teams in the NBA, this dataset will only have 30 observations as there will be one for each team, so it is important to avoid overfitting and drawing incorrect conclusions from the model with a low sample size.

```r
# Dataset: NBA 2025-2026 Advanced Team Statistics
# from: https://www.basketball-reference.com/leagues/NBA_2026.html
advanced_df <- read_csv("advanced_25_26.csv", skip = 1)
```

```
## New names:
## Rows: 31 Columns: 31
## -- Column specification
## -------------------------------------------------------- Delimiter: "," chr
## (2): Team, Arena dbl (26): Rk, Age, W, L, PW, PL, MOV, SOS, SRS, ORtg, DRtg,
## NRtg, Pace, FTr,... lgl (3): ...18, ...23, ...28
## i Use `spec()` to retrieve the full column specification for this data. i
## Specify the column types or set `show_col_types = FALSE` to quiet this message.
## * `` -> `...18`
## * `eFG%` -> `eFG%...19`
```

```
## * `TOV%` -> `TOV%...20`
## * `FT/FGA` -> `FT/FGA...22`
## * `` -> `...23`
## * `eFG%` -> `eFG%...24`
## * `TOV%` -> `TOV%...25`
## * `FT/FGA` -> `FT/FGA...27`
## * `` -> `...28`
```

```r
# clean column names for main predictors
# the second eFG and TOV percentages are for the opponents, so can ignore for the purposes of this proj
advanced_df <- advanced_df %>%
  rename(
    `eFG%` = `eFG%...19`,
    `TOV%` = `TOV%...20`,
  )
head(advanced_df)
```

```
## # A tibble: 6 x 31
##      Rk Team     Age     W     L    PW    PL   MOV   SOS   SRS  ORtg  DRtg  NRtg
##   <dbl> <chr>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1     1 Oklah~  25.2    41    13    44    10 11.9  -0.46 11.4   119.  107.  11.9
## 2     2 Detro~  25.9    39    13    38    14  7.77 -1.11  6.66  117.  109.   7.8
## 3     3 Bosto~  26.7    34    19    37    16  6.68 -0.89  5.79  121.  114.   7
## 4     4 New Y~  28.2    34    19    35    18  5.53 -0.49  5.04  120.  114.   5.7
## 5     5 San A~  25.6    36    16    34    18  5.4   0.14  5.54  117.  112.   5.3
## 6     6 Houst~  26.6    32    19    33    18  5.14 -0.17  4.97  118.  113    5.3
## # i 18 more variables: Pace <dbl>, FTr <dbl>, `3PAr` <dbl>, `TS%` <dbl>,
## #   ...18 <lgl>, `eFG%` <dbl>, `TOV%` <dbl>, `ORB%` <dbl>, `FT/FGA...22` <dbl>,
## #   ...23 <lgl>, `eFG%...24` <dbl>, `TOV%...25` <dbl>, `DRB%` <dbl>,
## #   `FT/FGA...27` <dbl>, ...28 <lgl>, Arena <chr>, Attend. <dbl>,
## #   `Attend./G` <dbl>
```

The context of this problem is that I want to answer the main question: Which performance metrics in the NBA advanced analysis data are the most essential to predict a winning season in the NBA? Specifically, the response variable is the Win/Loss percentage in a season that a team will have.

This situation requires interpretability because it allows for team managers in the NBA to know if their current roster is capable of succeeding or if they are just getting lucky with their schedule. The people that need this sort of information specifically are NBA head coaches and front office executives. Specifically, decisions about a team roster play an integral role in a team success, such as large salaries within contracts and player trades, so it is important to understand the specific metrics that increase success. My goal with this project is to find a complex model that will be able to uncover how these metrics actually influence success and potentially help teams with this decision making.

The main variables I will be testing are standardized versions of these metrics:

- Effective Field Goal % (eFG%)
- Turnover % (TOV%): Turnover to Total Possession ratio
- Offensive Rebounding % (ORB%): Offensive Rebounds to Total Rebounds ratio
- Net Rating (NRtg)
- Strength of Schedule (SOS)

The existing interpretable approaches include Linear Regression (where a few important variables of the game are chosen for a model) or formulas that base off of a team's points scored per game versus the points their opponents score per game. These sorts of methods are very easily interpretable because they focused on fixed weights, but the game of basketball is much more complex as these methods do not consider various interactions that are not linear (like having a high turnover rate can be balanced by having a high offensive rebounding rate). My approach uses LASSO (linear but using penalties) for variable selection and CART to showcase the complex interactions between these variables that previous methods are not able to find.

b) Method 1 Application

Background

Train an intrinsically interpretable model. Report:

- Out-of-sample prediction accuracy
- Interpretability output (e.g., variable importance, posterior samples, tree deci-sion rules)

Solution

For the first method, I use LASSO...

```
set.seed(43)

# selecting the main predictors and the response
predictors <- advanced_df %>%
  filter(Team != "League Average") %>%
  dplyr::select("eFG%", "TOV%", "ORB%", "NRtg", "SOS") %>%
  scale() %>%
  as.matrix()

response <- advanced_df %>%
  filter(Team != "League Average") %>%
  mutate(`Win%` = W / (W + L)) %>%
  pull("Win%")

# Cross-Validation for LASSO
cv_lasso <- cv.glmnet(predictors, response, alpha = 1, nfolds = 30)
```

```
## Warning: Option grouped=FALSE enforced in cv.glmnet, since < 3 observations per
## fold
```

```
# check RMSE
rmse <- sqrt(cv_lasso$cvm[cv_lasso$lambda == cv_lasso$lambda.min])

# coefficients
coeffs_lasso <- coef(cv_lasso, s = "lambda.min")

rmse; coeffs_lasso
```

```
##        s31
## 0.04941964
```

```
## 6 x 1 sparse Matrix of class "dgCMatrix"
##              lambda.min
## (Intercept) 0.50030457
## eFG%        0.01168379
## TOV%        .
## ORB%        .
## NRtg        0.13071021
## SOS         .
```

For the cross-validation piece of the LASSO method, I used 30 separate folds for Leave-One-Out Cross-Validation for every one of the 30 teams to be used as a test set, which will work well with my small sample size.

The out-of-sample prediction accuracy is the RMSE of about 0.0494, so the typical prediction error is about 4.94 percentage points away from the true result.

Now, based on the LASSO model, we can see which features are the most important for predicting the win percentage. The two coefficients that did not reach 0 are NRtg and eFG%, so the model assumes that these variables are necessary and perform the best. As for the interpretation for NRtg, for every 1 standard deviation increase, the win percentage is predicted to go up by 0.1307 or 13.0710 percentage points (stronger predictor). For eFG%, for every 1 standard deviation increase, the win percentage is predicted to increase by 0.01168. The coefficients that were 0 (which was for the variables TOV%, ORB%, and SOS) tells us that these variables were excluded by the LASSO regularization. Specifically, this tells us that once NRtg and eFG% were included in the model, the other variables did not provide any additional predictive performance.

c) Method 2 Application

Background

Repeat with a different intrinsically interpretable model.

Solution

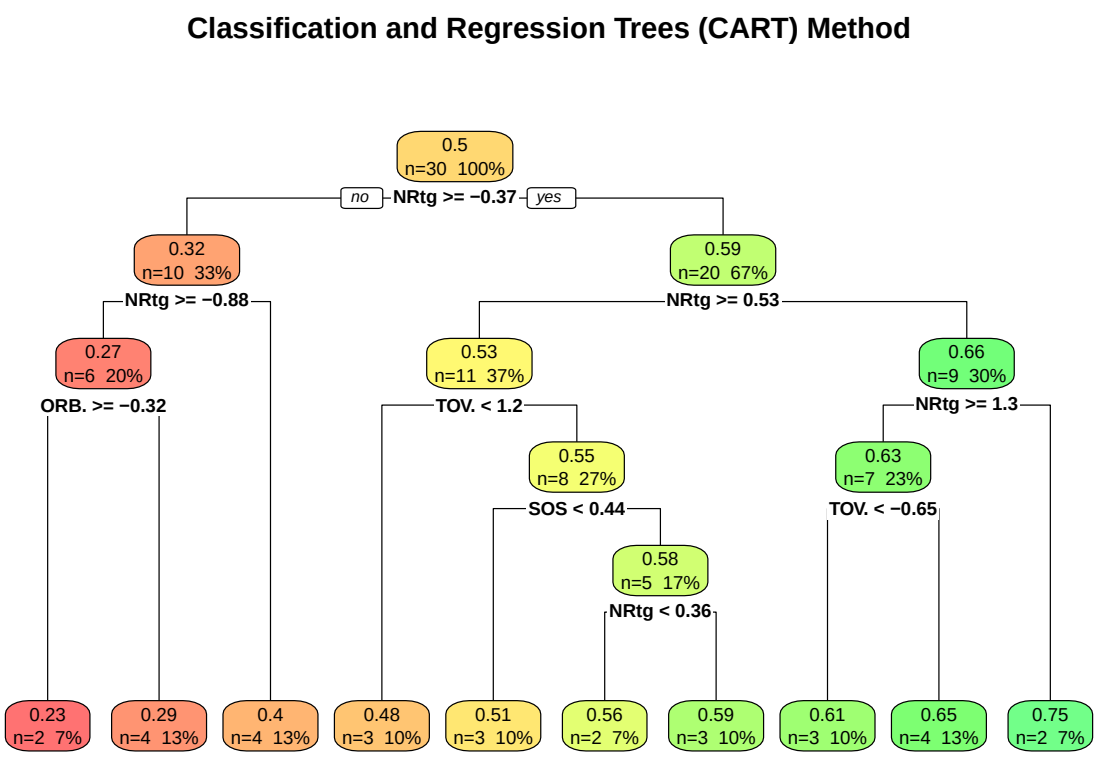
For the second method, I implemented a Classification and Regression Tree (CART) method that focuses more on the hierarchy rather than the linearity of LASSO. This is also much easier to follow through a flowchart and easily interpretable for a human. In this case, since the Win % response variable is continuous, my implementation of CART uses a regression tree with the method "anova".

```r
set.seed(99)

# df for tree for rpart
tree_df <- data.frame(predictors, Win_Perc = response)

# finding LOOCV RMSE
n <- nrow(tree_df)
predictions_loocv <- numeric(n)

for (t in 1:n) {
  train <- tree_df[-t, ]
  test <- tree_df[t, , drop = FALSE]
  tree_full_model <- rpart(Win_Perc ~ .,
                           data = train,
                           method = "anova",
```

```
                    control = rpart.control(cp = 0.001, minsplit = 5))
  predictions_loocv[t] <- predict(tree_full_model, newdata = test)
}

# RMSE
rmse_f_tree <- sqrt(mean((tree_df$Win_Perc - predictions_loocv)^2))
rmse_f_tree
```

```
## [1] 0.07622743
```

```
# visualize the tree
built_tree <- rpart(Win_Perc ~.,
                    data = tree_df,
                    method = "anova",
                    control = rpart.control(cp = 0.001, minsplit = 5))
rpart.plot(built_tree, type = 2, extra = 101, left = FALSE, main = "Classification and Regression Trees
```



For the out-of-sample prediction accuracy, I calculated the leave-one-out cross-validation RMSE which was about 0.0762. This shows that the tree's predictions typically deviated from the true Win % by an average of about 7.62 percentage points.

Unlike LASSO (which uses additive linear effects), the partitions of the regression tree are split into regions by specific decisions rules. So, in this case, we can see the importance of variables like TOV%, ORB% and SOS and how they depend on the value of NRtg, which reveals the complex, conditional relationships that we did not see with LASSO.

For the tree, the visualization starts with the most important split which is the NRtg (Net Rating). Before continuing, it is important to note that the green color represents the better winning percentages, the red represents the worse winning percentages, and the yellow represents the middle ground. For the NRtg split, any team with a NRtg above -0.37 are classified as good. In this case, the tree also uses ORB%, TOV%, and SOS as predictors having non-linear interactions depending on the split of NRtg. For example, for the ORB%, for teams that are not well rated (have a very low NRtg) as long as they have an ORB% of over -0.32 (as like a safety final resort), then their win percentage increases. The perfection standard (best case scenario) would be for teams that have a NRtg greater than 1.3, meaning they are essentially dominant in the league. Overall, as you work throughout the tree depending on the team's statistics, we are able to get a team's win percentage and the category that they are in relative to the league's average.

I also wanted to mention my use of the value 0.001 as my complexity parameter, which I used specifically in order to allow the model enough flexibility while also making sure that the splits made were meaningful with having a smaller sample size. In this case, instead of selecting the complexity parameter right from the pruning table, I fixed the value to 0.001 and then calculated the out-of-sample RMSE using leave-one-out cross-validation. This allows the tree to continually grow without overfitting and assess the predictive accuracy on the observations that were left out. My goal here was to balance interpretability and flexibility of the model since I have a very small sample size of only 30 teams.

d) Comparison

Background

Which conclusions persist across methods? Which differ, and why might this occur? Include 1 - 2 figures to justify your conclusions.

Solution

Between the LASSO and CART methods, there were similarities and also significant differences in the conclusions that both came to.

Similarities:

First of all, the main similarity was that both models reached the conclusions that NRtg (Net Rating) was the most important predictor of a winning season. In the LASSO model, we saw this through seeing that NRtg had the largest positive coefficient and in the CART model, it was the primary split. This shows consistency for the NRtg predictor as two models that are very fundamentally different provide a similar conclusion, so we know there is a strong association between the NRtg predictor and Win %.

Differences:

For the differences, the LASSO model had a lower out-of-sample RMSE versus the CART model (0.0494 and 0.0802 respectively). So, these tell us that the LASSO model had a better overall accuracy which is also more likely because of the small sample size of only 30 teams, allowing LASSO to be more stable and have less of a chance to overfit. This most likely occurs because LASSO has the shrinkage piece that allows for reduced variance with such a small sample size, but CART can have a higher variance due to the repetitive partitioning of only 30 samples.

In addition, another major difference was how both methods came to the conclusion of significant predictors. With LASSO following more of a linear path and shrinking coefficients of TOV%, ORB%, and SOS to 0 under regularization which showcased to us that in this additive linear framework, these three variables did not improve predictive performance when the other two predictors (eFG% and NRtg) are already accounted for. On the other hand, CART was able to identify the hidden complex relationships that go beyond linearity (as I described initially in part a) in basketball. The CART tree showed that although the "ruled-out" predictors may not have a significant linear impact, they do have a situational impact. This was shown (for example)

with the ORB% which acted as a safety net for teams that had a low NRtg, allowing us to not rule out that they are a bad team right away.

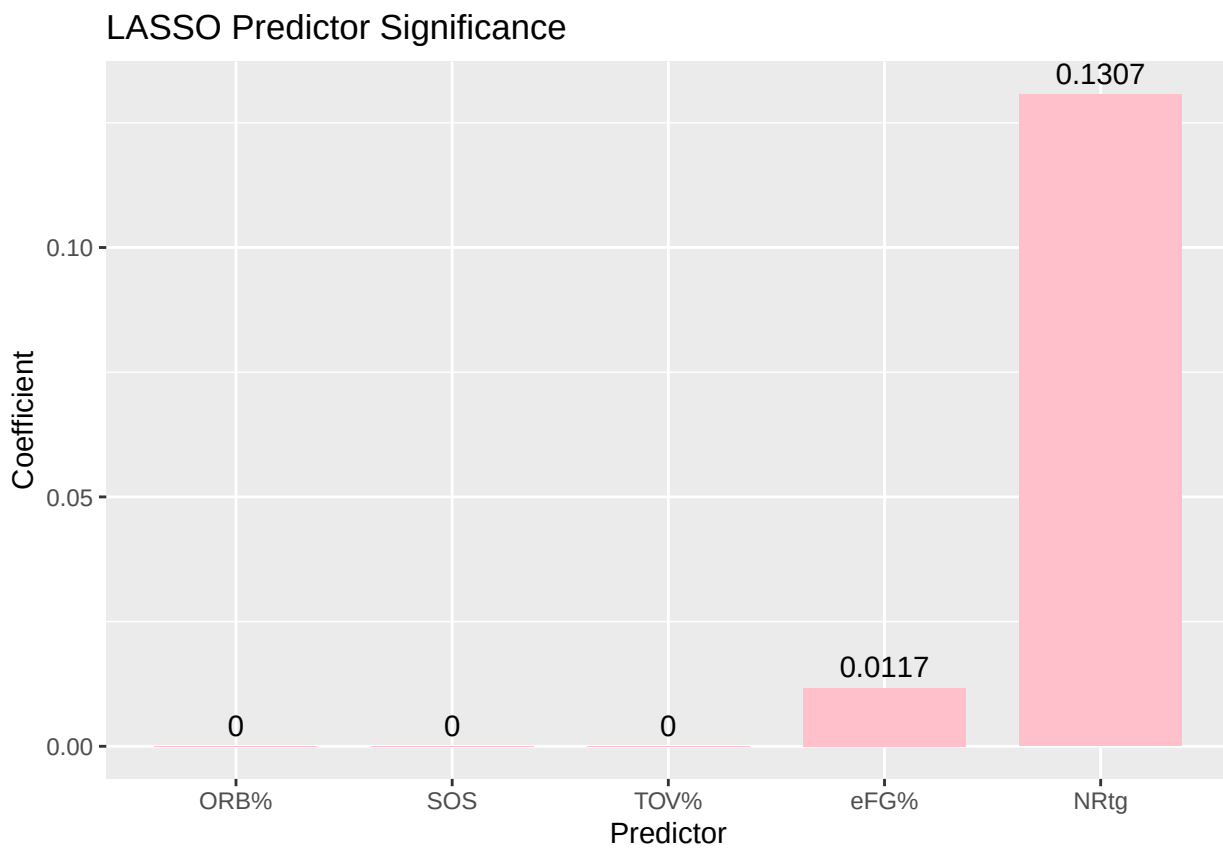
These differences occur specifically due to the underlying concepts for each model. Specifically, while LASSO uses more of an additive relationship as it takes into account each predictor, the CART method focuses more on conditional relationships, where the importance of a predictor changes depending on what the other statistics are. Additionally, LASSO focuses more on the linear relationships in an additive manner, which lead to it not using any predictors that caused little to no improvement in the predictive accuracy under regularization. CART focused more on partitioning in a repetitive manner in order to uncover hidden, complex relationships where predictors may depend on one another.

In conclusion, while LASSO provides a more stable predictive model (in my case with a smaller sample size), CART provides more interpretability based on conditions. This exemplifies how different modeling techniques can singling out different important pieces of the same dataset.

Important Figures:

For the CART method, the classification tree in part c provides the best visual explanation for how to classify teams by their success based on the predictors I provided from my basketball knowledge. However, for the LASSO method, I provided a bar graph below to showcase how this method expressed 3 predictors are redundant by shrinking their coefficients to 0 (classifying them as unnecessary):

```r
# convert to data frame + clean up
lasso_df <- as.data.frame(as.matrix(coeffs_lasso))
lasso_df$Variable <- rownames(lasso_df)
colnames(lasso_df)[1] <- "Prediction"

# taking away the intercept
lasso_no_int <- lasso_df[lasso_df$Variable != "(Intercept)", ]

# plot
ggplot(lasso_no_int, aes(x = reorder(Variable, Prediction), y = Prediction)) +
  geom_col(fill = "pink", width = 0.75) +
  geom_text(aes(label = round(Prediction, 4)), vjust = -0.5) +
  labs(title = "LASSO Predictor Significance",
       x = "Predictor",
       y = "Coefficient")
```



e) Sanity Check

Background

Test the reliability of one model. Acceptable checks:

- Vary hyperparameters; verify key conclusions hold
- Permute labels (null control)
- Bootstrap data; check stability

Solution

I am testing the reliability of my first model, the LASSO model. To do this, I varied the hyperparamater to see if the two predictors that were shown to be significant earlier (NRtg and eFG%) are still dominant.

Changing the hyperparameter from λ_{min} to λ_{1se}:

```
# getting coefficients for lambda_1se
onese_coeffs <- coef(cv_lasso, s = "lambda.1se")
onese_coeffs
```

```
## 6 x 1 sparse Matrix of class "dgCMatrix"
##              lambda.1se
```

```
## (Intercept) 0.500304565
## eFG%        0.002274067
## TOV%        .
## ORB%        .
## NRtg        0.121310395
## SOS         .
```

With these coefficients, it is clear that NRtg is still the most important predictor as it's coefficient is still the largest coefficient (although it decreased slightly from about 0.1307 to 0.1213) even when given a more strict penalty. Additionally, the predictor eFG% is still considered significant although very close to being insignificant. It's weight now, about 0.0023, was also lowered from initially being about 0.0117. The other three predictors that were initially considered insignificant have the same status now, as the coefficients for TOV%, ORB%, and SOS were reduced to 0 with the new hyperparameter.

As to why this is happening, λ_{1se} applies a stronger penalty and results in a more sparse model after the regularization process. So, when I got the same results of NRtg and eFG% being selected through the new process, this exemplifies that they truly are strong predictors. If the predictors truly were not strong, this new, stronger penalty would have shrunk the coefficients to 0 or chosen different predictors entirely. Through performing this final sanity check, it is definitely clear that the results I achieved initially are not just a random occurrence but important outcomes to truly aid in better predicting NBA win percentages.